

Architecture Checkpoint Report

Assignment 2: Architectural Evolution of nopCommerce

Group: 107

Members: Afonso Ferreira (113480), Tomás Brás (112665),
Hugo Ribeiro (113402), Rodrigo Abreu (113626)

April 27, 2026

Contents

1	Introduction and Scenario	1
1.1	Scenario Choice	1
1.2	Core Architectural Problem	1
2	Current-State Analysis	2
2.1	Architecture Overview	2
2.2	How a Purchase Works Today	3
2.3	What Already Works for Scenario C	3
2.3.1	Order Capture and Customer Visibility	3
2.3.2	Inventory and Warehouse Model	3
2.3.3	Fulfillment and Shipping	3
2.3.4	Plugins and Scheduled Tasks	4
2.4	What Does Not Work for Scenario C	4
2.4.1	No Durable Integration Path	4
2.4.2	No Freshness or Source Metadata on External State	4
2.4.3	Synchronous Plugin Calls Expose the Customer to External Failures	4
2.4.4	No Standard Failure State or Reconciliation Model	4
2.4.5	Shared Database Must Stay Internal	4
2.5	Useful Seams for Evolution	5
2.6	What This Means for the Target Architecture	5
3	Domain and Bounded Contexts	6
3.1	Domain Overview	6
3.2	Bounded Contexts	6
3.3	Responsibilities	6
3.4	Architecture Diagram	6
4	Quality Attribute Scenarios	7
5	Architectural Approach	8
5.1	Chosen Framework	8
5.2	Justification	8
6	Target Architecture	9
6.1	Overview	9
6.2	Components	9
6.3	Interactions	9
6.4	Data Ownership	9
6.5	Main Architecture Diagram	9

7	Architectural Decisions	10
7.1	ADR 1 - XXX	10
7.1.1	Context	10
7.1.2	Decision	10
7.1.3	Alternatives Considered	10
7.1.4	Consequences	10
7.2	ADR 2 - XXX	10
7.2.1	Context	10
7.2.2	Decision	10
7.2.3	Alternatives Considered	10
7.2.4	Consequences	10
7.3	ADR 3 - XXX	10
7.3.1	Context	10
7.3.2	Decision	10
7.3.3	Alternatives Considered	10
7.3.4	Consequences	10
8	Risk and Validation Plan	11
9	Evolution Roadmap	12
10	Feasibility Spike	13
10.1	What Was Tested	13
10.2	What Worked	13
10.3	Evidence	13

Chapter 1

Introduction and Scenario

1.1 Scenario Choice

This project follows **Scenario C - Omnichannel Commerce Core**. VerdeMart Retail started by using nopCommerce as its online storefront, but the business now needs more than a web shop. It wants web sales, warehouse execution, shipping, store operations, support, and customer visibility to work together as part of one commerce experience.

In this scenario, nopCommerce becomes the commerce core of that wider ecosystem. A web order should be reflected in operational systems such as warehouse and shipping, and changes that happen outside nopCommerce, such as stock or fulfillment updates, should become visible again in the customer and operator experience. This is important because those surrounding systems may be slow, stale, unavailable, or inconsistent, but the commerce experience still needs to remain useful and understandable.

1.2 Core Architectural Problem

Currently, nopCommerce mainly acts as the online shop: it owns the storefront, catalog browsing, cart, checkout, customer accounts, and order creation inside one web-centered platform. This works while the business is mostly operating through the online channel, but it does not fully address an omnichannel environment where warehouse execution, shipping, store operations, and support systems also influence the customer experience.

The core architectural problem is deciding how this online shop can evolve into a commerce core without assuming that every surrounding operational system is always fast, available, and consistent. The architecture must define what remains inside nopCommerce, what moves around it, and how the system behaves when warehouse or shipping state is delayed, stale, unavailable, or later recovered.

Chapter 2

Current-State Analysis

2.1 Architecture Overview

nopCommerce follows onion architecture, where all dependencies point inward toward the core. The solution is split into four main layers: **Nop.Core**, **Nop.Data**, **Nop.Services**, and the presentation layer (**Nop.Web** and **Nop.Web.Framework**). NopcommerceGitHub Each layer has a

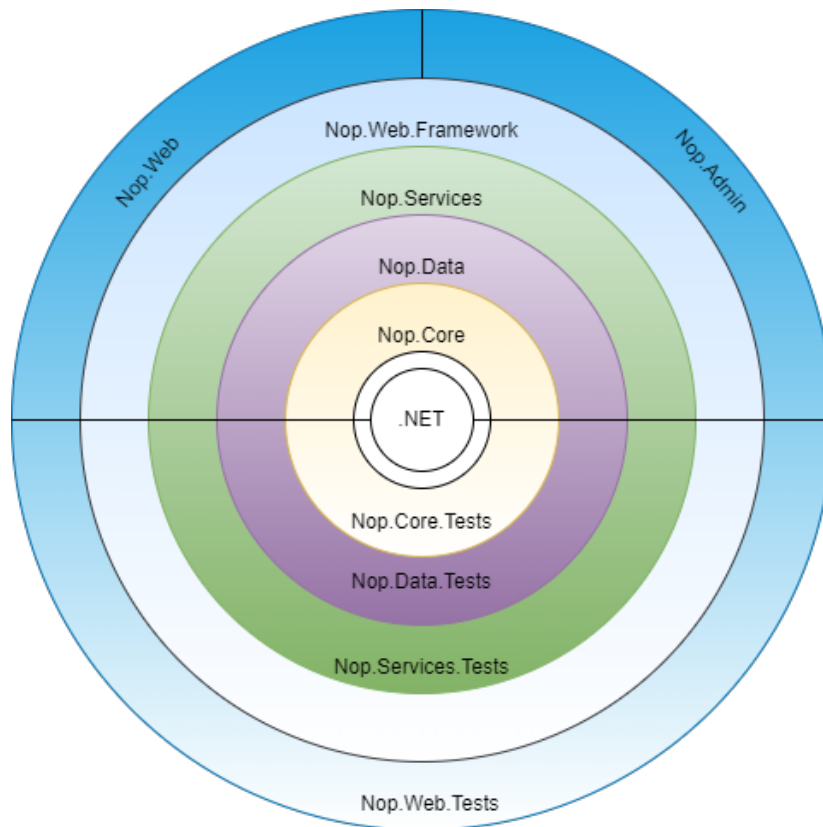


Figure 2.1: Current layered/onion architecture of nopCommerce.

specific role:

- **Nop.Core:** The innermost layer. Contains domain entities, caching, events, and helper classes shared across the whole solution. It has no dependencies on any other project. Nop-commerce Key Scenario C entities live here: **Order**, **OrderItem**, **Product**, **ProductWarehouseInventory**, **StockQuantityHistory**, **Warehouse**, and **Shipment**.
- **Nop.Data:** Separates data-access logic from business objects. nopCommerce uses a Linq2DB

Code-First approach, with FluentMigrator for schema migrations and Fluent API for persistence mappings. nopcommerce

- **Nop.Services:** The outermost layer of the Application Core. It holds business logic, validations, and calculations, and uses a repository pattern to expose features to layers above it. [GitHub](#) This is where order processing, inventory adjustment, shipping, and checkout logic live.
- **Nop.Web** and **Nop.Web.Framework:** The presentation layer. **Nop.Web** contains controllers, models, views, and the storefront and admin area. **Nop.Web.Framework** is a shared class library with common presentation logic used by both the public site and the admin panel. nopCommerce
- **Plugins:** Loaded as separate Visual Studio projects and extend nopCommerce for payments, shipping, authentication, search, and other integrations. nopcommerce They run inside the same runtime as the monolith.

2.2 How a Purchase Works Today

A customer order flows from the storefront controllers in **Nop.Web** down through **Nop.Services**. **OrderProcessingService** creates the order, adjusts inventory, persists state via repositories, and publishes internal events like **OrderPlacedEvent**. Shipment updates follow later through shipment services, which update order status and trigger notification events. This works cleanly when nopCommerce owns the full picture. It breaks down when an external warehouse, POS, or carrier system owns part of the order's lifecycle and may be slow, unavailable, or return conflicting data. The internal event system is also worth noting. It publishes events in-process to registered consumers and awaits each one. If a consumer fails, the error is logged and execution continues. There is no message broker, no retry mechanism, no replay, and no dead-letter queue. This is fine for cache invalidation or plugin reactions — it is not sufficient for reliable integration with external systems.

2.3 What Already Works for Scenario C

2.3.1 Order Capture and Customer Visibility

nopCommerce has mature order management: customer accounts, checkout, payment status, order history, shipment tracking, admin screens, and notification emails. This gives Scenario C a solid foundation. The order placement workflow is a natural seam where an integration event like "order accepted for fulfillment" can be emitted — currently it only fires in-process, but extending it with a durable outbox is straightforward without touching checkout itself.

2.3.2 Inventory and Warehouse Model

The platform already models stock quantity, reserved quantities, warehouse assignments, and stock history. This is directly useful for omnichannel stock visibility. The limitation is that nopCommerce treats this as local application state — it has no concept of whether a stock number came from the WMS ten seconds ago or ten hours ago, or whether a POS sale has already reduced it.

2.3.3 Fulfillment and Shipping

Shipment entities, tracking numbers, shipping plugins, pickup-in-store support, and customer notifications for shipping progress are all present. These support buy-online-fulfill-elsewhere

flows. The gap is that shipment state is currently set by administrators or computed inside nopCommerce — there is no first-class integration status like "sent to WMS," "warehouse delayed," or "awaiting carrier confirmation."

2.3.4 Plugins and Scheduled Tasks

The architecture is modular, making it compatible with a variety of third-party services including ERPs, PIMs, and POS systems through its Web API plugin. Wired In Scheduled tasks can run background work — retries, synchronization, cleanup — and are protected against overlapping execution. These are useful extension points for Scenario C, though they lack the reliability structure needed for durable integration on their own.

2.4 What Does Not Work for Scenario C

2.4.1 No Durable Integration Path

The biggest gap is that there is no outbox, no integration message lifecycle, and no retry state. When an order is placed and the warehouse system is down, nopCommerce has no built-in guarantee that a fulfillment command was recorded, will be retried, or can be reconciled later. Logging an error is not a recovery path — operators need to know whether the order is waiting, failed, duplicated, or already accepted.

2.4.2 No Freshness or Source Metadata on External State

Stock, shipment, and fulfillment data can come from WMS, POS, ERP, or manual admin updates. The current model stores a value but does not record where it came from, when it was last confirmed, or whether it conflicts with another source. Stale data looks identical to fresh data to both customers and operators.

2.4.3 Synchronous Plugin Calls Expose the Customer to External Failures

When a plugin calls an external system during checkout or shipping-rate calculation, a slow or failed response degrades the customer experience directly. Scenario C requires the commerce core to stay useful when surrounding systems degrade — accepting safe work, signaling degraded status, and deferring unsafe steps — rather than failing visibly.

2.4.4 No Standard Failure State or Reconciliation Model

There is no common status model for integration messages: no "pending," "retrying," "failed," or "dead-lettered" states. There is no correlation between a placed order, the fulfillment command sent from it, and the acknowledgement (or conflict) that comes back. This makes recovery from partial failures opaque.

2.4.5 Shared Database Must Stay Internal

The nopCommerce database blends catalog, orders, customers, stock, shipments, configuration, and logs. This is fine for a monolith. It becomes a problem if external adapters start reading or writing it directly. Presentation logic in `Nop.Web` is isolated from the database — controllers only reference services, not repositories directly. nopCommerce The same discipline must apply at system boundaries: any fulfillment adapter, stock projection, or integration worker must communicate through APIs or events, not by coupling to nopCommerce tables.

2.5 Useful Seams for Evolution

The architecture provides clear attachment points without requiring a rewrite:

- **Order placement:** emit a durable integration message for fulfillment and ERP immediately after an order is accepted.
- **Shipment updates:** receive warehouse and carrier progress and update nopCommerce shipment projections and customer-visible status.
- **Inventory:** receive stock changes from WMS or POS and update a customer-facing availability view with freshness metadata.
- **Scheduled tasks:** run outbox publishing, retry logic, timeout detection, and reconciliation outside the customer request path.
- **Plugins and APIs:** connect to surrounding systems through explicit adapters, not direct database access.
- **Admin visibility:** expose integration status, failures, and reconciliation actions to operators.

2.6 What This Means for the Target Architecture

nopCommerce should remain the owner of checkout, order management, customer visibility, catalog, and administration. Moving those out would create unnecessary scope. The evolution targets three specific gaps instead:

- **Reliable propagation:** fulfillment and ERP commands must leave through a durable asynchronous path, not in-process events or direct plugin calls.
- **Visible degradation:** when WMS, POS, ERP, or carriers are slow, unavailable, or contradictory, the system should expose an honest state — pending, delayed, degraded, or reconciliation required — rather than silently serving stale data.
- **Recovery and reconciliation:** retries, idempotency, correlation identifiers, and operator-visible failure records should make it possible to recover without duplicating orders or hiding conflicts.

The current state is a strong starting point: it provides a real commerce base with genuine seams. The evolution adds reliability and cross-channel state visibility around the existing monolith rather than replacing it.

Chapter 3

Domain and Bounded Contexts

- 3.1 Domain Overview
- 3.2 Bounded Contexts
- 3.3 Responsibilities
- 3.4 Architecture Diagram

Chapter 4

Quality Attribute Scenarios

Chapter 5

Architectural Approach

5.1 Chosen Framework

5.2 Justification

Chapter 6

Target Architecture

6.1 Overview

6.2 Components

6.3 Interactions

6.4 Data Ownership

6.5 Main Architecture Diagram

Chapter 7

Architectural Decisions

7.1 ADR 1 - XXX

7.1.1 Context

7.1.2 Decision

7.1.3 Alternatives Considered

7.1.4 Consequences

7.2 ADR 2 - XXX

7.2.1 Context

7.2.2 Decision

7.2.3 Alternatives Considered

7.2.4 Consequences

7.3 ADR 3 - XXX

7.3.1 Context

7.3.2 Decision

7.3.3 Alternatives Considered

7.3.4 Consequences

Chapter 8

Risk and Validation Plan

Chapter 9

Evolution Roadmap

Chapter 10

Feasibility Spike

10.1 What Was Tested

10.2 What Worked

10.3 Evidence